Data Science Lab

Name: Vishal Rajesh Mahajan Experiment No: 9

Class: BE INFT B-1 Roll No: 1

To evaluate and compare the performance of different classification algorithms.

Import Required Libraries

```
import time
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
```

Load the dataset

```
iris = load_iris()
X, y = iris.data, iris.target
```

Split dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
```

Scale features (needed for LR, KNN, SVM)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Define classifiers

```
classifiers = {
    "Logistic Regression (LR)": LogisticRegression(random_state=42, solver='liblinear',
max_iter=200),
    "Naive Bayes (NB)": GaussianNB(),
    "K-Nearest Neighbors (KNN)": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree (DT)": DecisionTreeClassifier(random_state=42, max_depth=5),
    "Support Vector Machine (SVM)": SVC(random_state=42, kernel='linear')
}
results = {}
```

Train and evaluate

```
for name, model in classifiers.items():
    start_time = time.time()

    if name in ["Logistic Regression (LR)", "K-Nearest Neighbors (KNN)", "Support Vector Machine
(SVM)"]:
        X_train_data, X_test_data = X_train_scaled, X_test_scaled
    else:
        X_train_data, X_test_data = X_train, X_test

    model.fit(X_train_data, y_train)
    y_pred = model.predict(X_test_data)

    results[name] = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision (Weighted)': precision_score(y_test, y_pred, average='weighted'),
        'Recall (Weighted)': recall_score(y_test, y_pred, average='weighted'),
        'F1-Score (Weighted)': f1_score(y_test, y_pred, average='weighted'),
        'Time (s)': time.time() - start_time
    }
```

```
print(f"Trained {name}. Accuracy: {results[name]['Accuracy']:.4f}")
```

```
Trained Logistic Regression (LR). Accuracy: 0.8000
Trained Naive Bayes (NB). Accuracy: 0.9111
Trained K-Nearest Neighbors (KNN). Accuracy: 0.9111
Trained Decision Tree (DT). Accuracy: 0.9333
Trained Support Vector Machine (SVM). Accuracy: 0.9111
```

Display results in a properly labeled table

```
results_df = pd.DataFrame(results).T.sort_values(by='Accuracy', ascending=False)
print("\n--- Performance Comparison Table (Sorted by Accuracy) ---")
print(results_df.to_markdown(floatfmt=".4f"))
```

```
--- Performance Comparison Table (Sorted by Accuracy) ---
|                            |   Accuracy |   Precision (Weighted) |   Recall (Weighted) |   F1-Score (Weighted) |   Time (s) |
|:---------------------------|-----------:|-----------------------:|--------------------:|----------------------:|-----------:|
| Decision Tree (DT)         |     0.9333 |                 0.9444 |              0.9333 |                0.9327 |     0.0056 |
| K-Nearest Neighbors (KNN)  |     0.9111 |                 0.9298 |              0.9111 |                0.9095 |     0.0079 |
| Naive Bayes (NB)           |     0.9111 |                 0.9155 |              0.9111 |                0.9107 |     0.0081 |
| Support Vector Machine (SVM) |     0.9111 |                 0.9155 |              0.9111 |                0.9107 |     0.0069 |
| Logistic Regression (LR)   |     0.8000 |                 0.8056 |              0.8000 |                0.7980 |     0.0122 |
```